

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Completed Benchmark Report

Hands-Off Benchmark Practicum

Student Name	Kemelov Nurbolat
Student ID	230103027
Section	04-P
Hardware / CPU	11th Gen Intel Core i5-1155G7 @ 2.50 GHz (4 cores / 8 logical threads)
Numba Threading Layer	workqueue
Date	02 October 2026

Instructor: sufyan.mustafa@sdu.edu.kz

Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark the Monte Carlo simulation for π across different thread counts and evaluate speedup and parallel efficiency.

Benchmark Results

Threads	Time (s)	Speedup	Efficiency (%)	π Estimate
1	2.5768	1.00x	100.0	3.14161250
2	1.2604	2.04x	102.2	3.14161410
4	0.7981	3.23x	80.7	3.14160943
8 (Max)	0.5819	4.43x	55.4	3.14199480

Recorded values: $T_1 = 2.5768$ s; $T_{\text{max}} (8 \text{ threads}) = 0.5819$ s. Best measured speedup = 4.43x.

Example calculation: Speedup at 8 threads = $2.5768 / 0.5819 = 4.43x$; efficiency = $(4.43 / 8) \times 100 = 55.4\%$.

Technical Questions

A. Record your baseline T_1 (1 thread) and your maximum-thread runtime T_{max} .

$T_1 = 2.5768$ s. With the maximum available thread count (8), $T_{\text{max}} = 0.5819$ s. The runtime decreased substantially as threads were added, but scaling became sub-linear at higher thread counts.

B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before benchmark timings?

Numba uses just-in-time (JIT) compilation. The first call includes compilation, type specialization, runtime initialization, and parallel backend startup. If that first-call overhead were included, the measured time would not represent only the Monte Carlo computation. The warm-up compiles the function before the real benchmark, so subsequent timings measure execution rather than compilation.

C. Did parallel efficiency stay at 100% from 1 to max threads? What architectural factors explain degradation?

No. Efficiency was 100.0% at 1 thread, 102.2% at 2 threads, 80.7% at 4 threads, and 55.4% at 8 threads. The small value above 100% at 2 threads is a normal measurement effect caused by factors such as CPU turbo frequency, cache state, OS scheduling, and timing variability; it does not mean the machine permanently exceeds ideal linear scaling. At higher thread counts, shared-resource contention, scheduling overhead, cache and memory traffic, synchronization/reduction overhead, and the use of SMT/hyperthreading logical threads rather than additional physical cores reduce efficiency. The processor has 4 physical cores and 8 logical threads, so the 8-thread run relies on SMT and cannot normally double the 4-core performance.

D. Why does `inside_circle += 1` inside prange not corrupt the count? What reduction does Numba generate?

Numba recognizes the scalar update as a reduction pattern. Each worker accumulates a private partial value for `inside_circle`, and the partial values are combined at the end of the parallel region. Conceptually this corresponds to an OpenMP-style `reduction(+:inside_circle)`, which avoids multiple threads racing on one shared increment operation.

Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Render a 2500×2500 Mandelbrot fractal and compare row-parallel and column-parallel decomposition for a non-uniform workload.

Benchmark Results

Decomposition	Active Threads	Time (s)
Rows	8	1.9034
Columns	8	1.9076

Difference: Row-parallel was faster by 0.0042 s, approximately 0.22%. The difference is extremely small, so the two runs were effectively similar on this machine.

Technical Questions

A. Compare `t_rows` vs `t_cols`. Which axis decomposition completed faster and why?

The row-parallel version completed in 1.9034 s and the column-parallel version in 1.9076 s, so row-parallel was slightly faster. The difference was only 0.0042 s (about 0.22%), so it should not be over-interpreted. Row-oriented traversal is naturally aligned with NumPy C-contiguous row-major storage, while the Mandelbrot computation itself is very compute-heavy; therefore memory-layout advantages exist but are partly hidden by the large arithmetic workload and run-to-run timing noise.

B. How does NumPy's C-contiguous memory layout interact with cache lines when threads write using row-wise vs column-wise traversal?

In a C-contiguous 2D NumPy array, consecutive columns of the same row are adjacent in memory. In the row-parallel function, the inner loop changes `c`, so writes to `img[r, c]` are sequential and cache-friendly. In the column-parallel function, the inner loop changes `r` while `c` is fixed, so successive writes are separated by an entire row stride. That pattern has weaker spatial locality, uses cache lines less efficiently, and can increase cache/TLB traffic. It can also create more cache-coherence pressure when multiple threads work on nearby columns.

C. Why can static row assignment create load imbalance, and how does dynamic scheduling resolve it?

Mandelbrot pixels do not all require the same number of iterations. Pixels far outside the set escape after only a few iterations, while pixels near or inside the set may execute close to `max_iter`. If a static partition gives one thread mostly easy rows and another thread mostly difficult central rows, the first thread finishes early and becomes idle while the second continues working. Dynamic scheduling breaks the iteration space into smaller chunks and assigns new chunks to workers as they become free, improving utilization and reducing the time lost to imbalance. The trade-off is extra scheduling overhead.

D. Attach the exported `mandelbrot_output.png` file.

The generated Mandelbrot artifact is included below and is also supplied as a separate PNG file with this report.

Mandelbrot 2500x2500 (Measured Render: 1.9034 s)

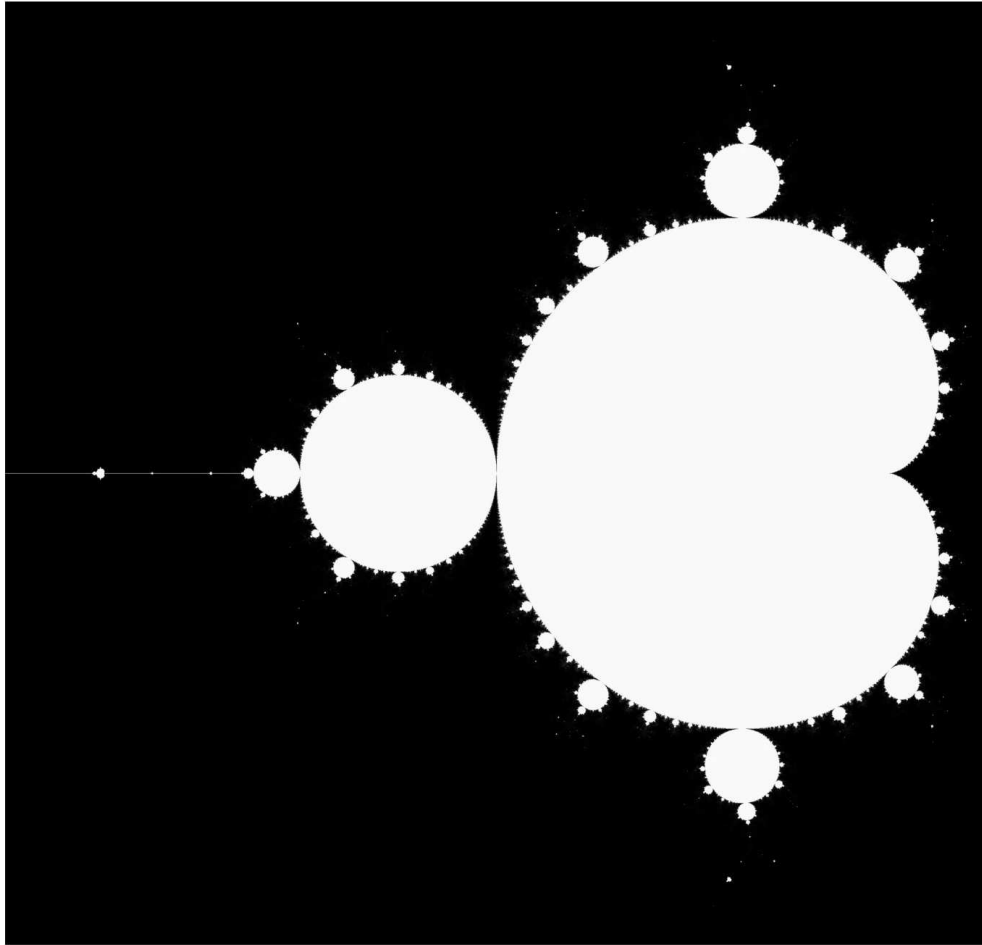


Figure 1. Mandelbrot 2500×2500 output ($MAX_IT = 1000$).

Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Run a 5-point heat-diffusion stencil on a 1500×1500 grid for 300 timesteps and evaluate the memory-bandwidth limit.

Benchmark Results

Precision	Threads	Runtime (s)	Throughput (Megacells/s)
float64	8	0.4815	1401.92
float32	8	0.2478	2724.16

Runtime decrease factor: $0.4815 / 0.2478 = 1.943x$. Throughput increased from 1401.92 to 2724.16 Megacells/s (about 1.94x).

Technical Questions

A. Record benchmark runtime and Megacells/sec throughput.

Using the required float64 arrays, runtime was 0.4815 s and throughput was 1401.92 Megacells/sec. With float32, runtime was 0.2478 s and throughput was 2724.16 Megacells/sec.

B. After changing np.float64 to np.float32, by what factor did runtime decrease, and why?

The runtime decreased by 1.943x ($0.4815 / 0.2478$). A float32 value occupies 4 bytes instead of 8 bytes, so the stencil moves roughly half as much array data per cell update. Because this computation repeatedly reads neighboring cells and writes a result, memory traffic dominates the cost. Reducing the bytes transferred allows more cell updates per second, improves cache capacity/effective locality, and more closely matches the available memory bandwidth. The improvement therefore comes mainly from reduced memory traffic rather than from a large change in arithmetic capability.

C. Why does doubling CPU cores fail to give 2x speedup after memory bandwidth is saturated?

The Roofline model states that performance is limited either by compute throughput or by memory bandwidth. The heat stencil has low arithmetic intensity: only a modest number of arithmetic operations are performed for several memory loads/stores. Once the active cores collectively consume the maximum sustainable RAM bandwidth, adding more cores cannot deliver data faster. Extra cores then compete for the same memory bus, so speedup flattens and may even decline slightly because of contention and parallel overhead.

Student Result Scorecard & Data Collection

Table 1 reproduces the lab scorecard using the measured benchmark results.

Benchmark Challenge	Active Threads	Array Size / Samples	Time (s)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	2.5768	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	1.2604	2.04x	102.2%
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.7981	3.23x	80.7%
Challenge 1: Monte Carlo	8 Threads (Max)	120,000,000	0.5819	4.43x	55.4%
Challenge 2: Mandelbrot (Rows)	8 Threads (Max)	2500 × 2500	1.9034	N/A	N/A
Challenge 2: Mandelbrot (Cols)	8 Threads (Max)	2500 × 2500	1.9076	N/A	N/A
Challenge 3: Heat Stencil	8 Threads (Max)	1500 × 1500 × 300	0.4815	N/A	N/A

Challenge 3 throughput (required separately by the task): 1401.92 Megacells/sec using float64. The float32 re-run achieved 2724.16 Megacells/sec with a 1.943x runtime decrease.

Overall Observation

The benchmarks show three different scaling behaviors. Monte Carlo scales strongly but loses efficiency when moving from physical cores to SMT logical threads. Mandelbrot exposes irregular per-pixel work and potential load imbalance, although row and column timings were almost identical in this run. The heat stencil is memory-bandwidth bound, and reducing the array precision from float64 to float32 nearly doubled throughput because the memory traffic per cell was approximately halved.